

මාදිලි පුහුණු කිරීම (Model Training)

Author: Yasithzz | 2026

01 -- මාදිලි පුහුණු කිරීම (Model Training)

LifeMate වියාපෘතිය -- සිංහල ලේඛනය

1. මාදිලි පුහුණු කිරීම යනු කුමක්ද?

"Model Training" කියන්නේ පරිගණකයට ඉගෙනීමට සිදු කිරීමේ ක්රියාවලියකි. අපි ලිහිල් භාෂාවෙන් කිවහොත් -- අපට ඕනෑම දේශීය ඉගෙනගන්නා ආකාරය ගැන සිතන්න. හුදුවරයා නිර්මාණය කරන දත්ත (examples) ගොඩනඟා පෙන්වූ විට, ශ්‍රමයා ඒ pattern රටාව හඳුනාගෙන ඊළඟ ප්රශ්නවලට නිවැරදි පිළිතුරු දෙයි. ඒ ආකාරයටම, LifeMate යෙදුමේදී, stress_dataset.csv ගොනුවෙන් ගත් දත්ත භාවිතා කරමින් RandomForestClassifier ඇල්ගොරිතම ඉගෙන ගනී -- ඉන්පසු නව පරිශීලකයෙකුගේ දෛනික දත්ත ලැබූ විට ඔහුගේ/ඇයගේ ආතතිය (stress level) කිහිකීමේ හැකියාව ලැබේ.

2. train.py ගොනුව -- රේඛා රේඛා පැහැදිලි කිරීම

2.1 -- ලේඛන ශීර්ෂය (Docstring)

```
"""
LifeMate Stress-Level Model Trainer
Trains a RandomForestClassifier on stress_dataset.csv.
Run: python train.py
Saves model/stress_model.pkl and model/model_meta.json.
"""
```

සිංහල: මෙය ගොනුවේ විස්තරයකි. RandomForestClassifier ඉගෙනවු පසු ලබාගත් ශික්ෂිත මාදිලිය stress_model.pkl ලෙස සහ තොරතුරු model_meta.json ලෙස සුරකිය.

2.2 -- කුළු ගවේෂ (Imports)

```
import os, json, joblib
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, accuracy_score
from imblearn.over_sampling import SMOTE
```

සිංහල: මගේ විවිධ Python library (සහාය ඒකක) ගනො ගැනීමකි.

Import	පිරයෝජනය
`os, json, joblib`	ගොනු කළමනාකරණය, JSON සුරකීම, Model සුරකීම
`pandas` (`pd`)	CSV ගොනුවෙන් දත්ත කියවීම සහ සැකසීම
`numpy` (`np`)	සංඛ්‍යා ගණනය සහ arrays සෑදීම
`RandomForestClassifier`	ශික්ෂිත කිරීමේ ඇල්ගොරිතම
`train_test_split`	දත්ත ශික්ෂණ හා පරීක්ෂා කොටස් ලෙස බෙදීම
`cross_val_score`	නිරවද්‍යතාව ගණනය කිරීම
`accuracy_score`	නිවැරදි රේඛා ශ්රේණිය
`SMOTE`	අසම දත්ත සමතුලා කිරීම

2.3 -- මාර්ග (Paths) හා නියතයන් (Constants)

```
BASE = os.path.dirname(__file__)
DATA = os.path.join(BASE, "..", "datasets", "stress_dataset.csv")
MODEL_DIR = os.path.join(BASE, "model")
os.makedirs(MODEL_DIR, exist_ok=True)
```

සිංහල:

- * BASE -- train.py ගොනුව ඇති folder එකේ ස්ථානය.
- * DATA -- stress_dataset.csv ගොනුවේ ස්ථානය.
- * MODEL_DIR -- ශික්ෂිත model සුරුකන folder (ml-service/model/).
- * os.makedirs -- model/ folder නොතිබේ නම් ස්වයංකාරීයව සෑදීම.

```
FEATURES = ["mood", "workload", "sleep_hours", "energy_level",
            "social_interaction", "exercise_done", "screen_time_hours", "water_cups"]
TARGET = "stress_level"
LABELS = ["Very Low", "Low", "Normal", "High", "Very High"]
```

සිංහල:

- * FEATURES -- model ඉගෙනීමට භාවිතා කරන ලක්ෂණ (features) 8 ක් -- මේ 8 දෙනෙක දත්ත ශ්රේණීන් (inputs) වයි.
- * TARGET -- model විසින් පිළිතුරු දිය යුතු output: stress_level (ආතතිය).

* LABELS -- ආතතිය ශ්‍රේණිගත කිරීමේ 5 ශ්‍රේණි.

2.4 -- load_data() ශ්‍රේණිය

```
def load_data(path=DATA):  
    df = pd.read_csv(path)  
    X = df[FEATURES]  
    y = df[TARGET]  
    return X, y
```

සිංහල: CSV ගොනුව කියවා, X (inputs -- 8 features) සහ y (output -- stress_level) ලෙස බෙදා ගනී. X ද්‍රෝණි, y ප්‍රතිඵලයකි.

2.5 -- SMOTE සීමා නියතයන්

```
MIN_SAMPLES_FOR_SMOTE = 30  
MIN_SAMPLES_FOR_SPLIT = 10
```

සිංහල:

- * SMOTE භාවිතා කිරීමට අවම සාම්පල ගණන 30.
 - * Train/Test ලෙස දත්ත බෙදීමට අවම 10.
-

2.6 -- train() ශ්‍රේණිය -- ප්‍රධාන කොටස

```
def train(X, y):  
    n_total = len(X)  
    class_counts = dict(zip(*np.unique(y, return_counts=True)))  
    min_class = min(class_counts.values())
```

සිංහල:

- * n_total -- සම්පූර්ණ දත්ත ගණන.
 - * class_counts -- සෑම stress ශ්‍රේණියකම ඇති records ගණන.
 - * min_class -- කුඩාම ශ්‍රේණියේ record ගණන (SMOTE තීරණය කිරීමට).
-

```
if n_total >= MIN_SAMPLES_FOR_SMOTE and min_class >= 6:  
    smote = SMOTE(random_state=42)  
    X_res, y_res = smote.fit_resample(X, y)  
else:  
    X_res, y_res = X.values, y.values
```

සිංහල:

- * දත්ත 30+ ඇත් නම් සහ සෑම class එකක් 6+ ඇත් නම් SMOTE ක්‍රියා කරයි -- අඩු class ද සමාන ප්‍රමාණයට නිර්මාණය කරයි.
 - * ප්‍රමාණවත් නොවූයේ නම් දත්ත ඒ ආකාරයටම (raw) ශික්ෂණයට යොදාගනී.
-

```

if n_total >= MIN_SAMPLES_FOR_SPLIT:
    X_train, X_test, y_train, y_test = train_test_split(
        X_res, y_res, test_size=0.2, random_state=42, stratify=y_res)
    clf = RandomForestClassifier(
        n_estimators=200, max_depth=5, random_state=42, n_jobs=-1)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

```

සිංහල:

- * දත්ත 80% ශික්ෂණයට (training), 20% පරීක්ෂාවට (testing) ලෙස බෙදෙයි.
- * RandomForestClassifier සෑදීම: 200 ගස් (trees), ගස් ගැඹුර 5, CPU cores සියල්ල (n_jobs=-1).
- * clf.fit() -- model ඉගෙනෙයි.
- * clf.predict() -- test දත්ත නිරවද්‍ය කිරීම.
- * accuracy_score -- නිරවද්‍යතාව ගණනය.

```

cv_folds = min(5, min_class) if min_class >= 2 else 0
if cv_folds >= 2:
    cv = cross_val_score(clf, X_res, y_res, cv=cv_folds, scoring="accuracy")

```

සිංහල: Cross-validation -- model රිදීම ආකාරයෙන් (folds ලෙස) නැවත නැවත පරීක්ෂා කරයි. ශ්රේණිය ස්ථාවරව සහ නිරවද්‍යව ක්රියාත්මක ද යන්න සහතික කිරීමකි.

```

else:
    clf = RandomForestClassifier(
        n_estimators=100, max_depth=3, random_state=42, n_jobs=-1)
    clf.fit(X_res, y_res)
    acc = accuracy_score(y_res, clf.predict(X_res))

```

සිංහල: දත්ත 10 ට අඩු නම් (fallback) -- කුඩා model (100 ගස්, ගැඹුර 3) සියලු දත්ත පුහුණු කිරීමටත් පරීක්ෂා කිරීමටත් ම යොදාගනී.

2.7 -- save() ශ්රිතය

```

def save(clf, acc):
    model_path = os.path.join(MODEL_DIR, "stress_model.pkl")
    meta_path = os.path.join(MODEL_DIR, "model_meta.json")
    joblib.dump(clf, model_path)
    with open(meta_path, "w") as f:
        json.dump({"features": FEATURES, "labels": LABELS,
                  "accuracy": round(acc, 4)}, f, indent=2)

```

සිංහල:

- * stress_model.pkl -- ශික්ෂිත model ගොනුව (binary).
- * model_meta.json -- features, labels, accuracy -- API එකට දෙන meta-data.

2.8 -- ක්‍රියාත්මක කිරීම

```
if __name__ == "__main__":
    X, y = load_data()
    clf, acc = train(X, y)
    save(clf, acc)
```

සිංහල: python train.py ධාවනය කළ විට මෙය ස්වයංක්‍රීයව: (1) දත්ත කියවයි, (2) model ඉගෙනෙයි, (3) model සුරකී.

3. RandomForestClassifier යනු කුමක්ද?

ලිහිල් භාෂාවතෝ: Random Forest ක්රමය "ගස් ගොඩාකනේ වනාන්තරයක්" (forest of decision trees) සාදයි. සෑම "ගසක්" ම self-contained ලෙස ප්රශ්නය විසඳයි -- "ඔහුගේ නිදිමට ඉතා අඩු ද? ඔහු වියායාම නොකළා ද?" -- ජය ශ්රේණිය (stress level) ගස් බොහෝ ගණනක් ඡන්ද දෙන ශ්රේණිය වෙයි.

LifeMate හි:

- * 200 ගස් (n_estimators=200): 200 "ගස්" ඒකක ඡන්ද දී ශ්රේණිය ප්රතිදානය කරයි.
- * max_depth=5: ගස් ගැඹුර 5 ලෙස සීමා කළේ ඔවර්ෆිට් (overfitting) වළකිනවා.
- * random_state=42: ප්රතිවිපේදාව (reproducibility) සහතික කිරීමට නිශ්චිත seed.
- * n_jobs=-1: පරිගණකයේ සියලු CPU cores යවන ලෙස යොදාගනී -- වේගවත් ශික්ෂණය.

4. SMOTE යනු කුමක්ද, එය ඇයි?

SMOTE -- Synthetic Minority Over-sampling Technique.

ලිහිල් භාෂාවතෝ: ඔබේ dataset හි "Very High" stress ශ්රේණිය ඇති records ගණන 5 ක් නම්, "Very Low" stress records ගණන 100 ක් ද ඇත නම් -- model ශ්රේණිය "Very Low" ලෙස හැමවිටම ඡන්ද දෙනු ඇත (සබලත්වය). SMOTE ශ්රේණිවලට "විශාජ" (synthetic) records ගණන සෑදීමෙන් ශ්රේණි සමතුලා කරයි -- model සාධාරණ ලෙස ඉගෙනෙයි.

LifeMate හි: සෑම stress ශ්රේණියකටම (Very Low -> Very High) ශික්ෂිත ගණන සමකිරීමට SMOTE විශාජ නවකතා නිර්මාණය කරයි.

5. Features 8 -- ලක්ෂණ 8 ක් සිංහලෙන්

Feature	සිංහල	ප්‍රමාණ (Scale)
`mood`	ඔබේ මනෝභාවය / හැඟීම	1 (Stressed) -> 5 (Great)
`workload`	වැඩ බරය / load	1 (Very Light) -> 5 (Very Heavy)
`sleep_hours`	ගත් නිදිමත් ගණන	0 -> 12 පැය
`energy_level`	ශක්ති මට්ටම	1 (Exhausted) -> 5 (Very Energized)
`social_interaction`	සමාජ ගනුදෙනු	1 (None) -> 5 (Very Social)
`exercise_done`	ඒයායාම කළාද?	0 = නැත, 1 = ඔව්
`screen_time_hours`	screen (phone/PC) බලා ගිය කාලය	0 -> 14 පැය
`water_cups`	ගවුළු ප්‍රමාණය ගණන	0 -> 16 කෝප්ප

Target (ප්‍රතිඵලය): stress_level -- 0 (Very Low) -> 4 (Very High)

ලේඛනය -- LifeMate ව්‍යාපෘතිය, Yasithzz, 2026